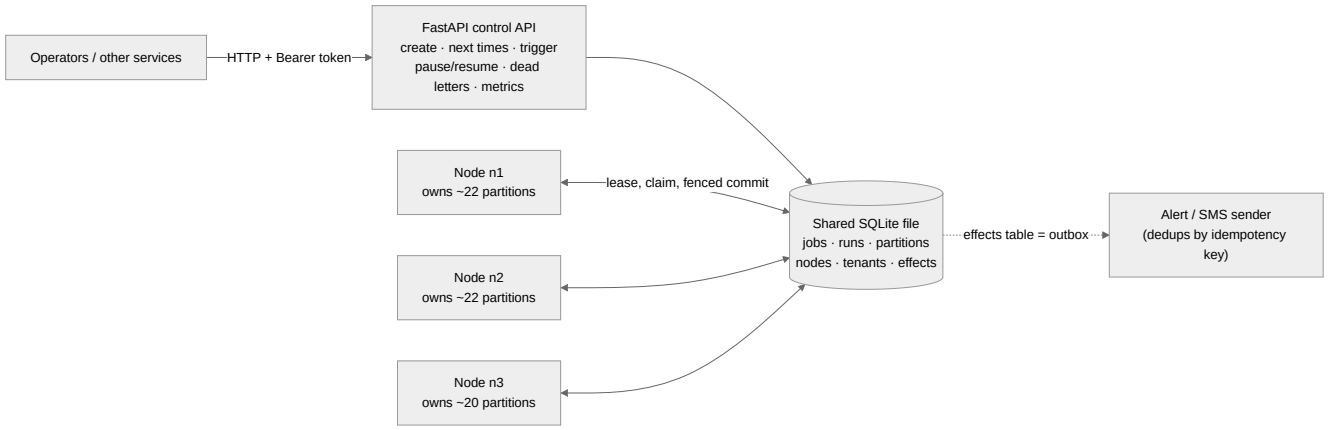
Distributed Trip Job Scheduler

MoveInSync SDE Backend Case Study 3: Exactly-Once Across a Cluster · Varun Mandava · Python 3 + FastAPI + SQLite

> **Result:** 3 real node processes ran **100,000 jobs due within 10 minutes with p99 fire lag 0.199 s** (target < 2 s). A node killed with `kill -9` while sending 1,200 pickup alerts: every employee got exactly one alert. A node frozen with `SIGSTOP` past its lease: another node finished the job, and the frozen node's late write was rejected by the fencing token. All 22 passed.

1. Architecture



- **Partitions (no single hot lock):** jobs map to 64 partitions by `crc32(job_id) % 64`. Each node holds a timed **lease** on a fair share (`ceil(64 / live_nodes)`) and only touches runs in its own partitions, so nodes never compete for the same rows. Leases are renewed 3× per lease period.
- **Rebalancing:** a node that joins makes the others' fair share smaller, so they hand back extra partitions. A node that dies stops heartbeating, its leases expire, and the others claim them.
- **Fencing token = partition epoch:** every change of owner does `epoch = epoch + 1`. A claimed run is stamped with that epoch, and `Store.finish()` writes the effect, the new status and the next occurrence in **one transaction** that first checks the epoch is still current.
- **Idempotency key:** `job_id:scheduled_time` (manual triggers get a unique key). The `effects` table is unique per key, so it works as an outbox.
- **Layers:** `api.py` (controller) → `node.py` (service) → `store.py` (repository). Job types are subclasses of the abstract `JobHandler` (polymorphism). The core logic (`timeutil`, `node`, `store`) is plain Python, tested without a web server.

2. The guarantees, answered

Why "exactly-once execution" is impossible. A node can perform the work and crash before it records that it did. From outside, "crashed before the work" and "crashed after the work" look the same, so the only safe choice is to run it again. So we guarantee **at-least-once execution** plus an **idempotency key**, which gives an **exactly-once effect**. The kill -9 test shows runs that executed twice (`attempts ≥ 2`) but produced one alert.

Which clock decides. The **database clock** (`julianday('now')` in SQLite, `now()` in Postgres). Due times, lease expiry and heartbeats are all read and compared inside the same transaction using that

one clock, so clock skew between nodes cannot make two nodes think they own the same partition. Node clocks are only used to time a handler's duration (`max_runtime`).

Daylight saving (cron is read in the tenant's time zone):

- **Gap.** A 02:30 job on the day clocks jump 02:00→03:00 runs at **03:00**, the instant the clock jumps.
- **Fold.** On the day 01:30 happens twice, the job runs **once, at the first 01:30**. Hourly jobs do not repeat the folded hour. Both cases are tested with real 2026 New York dates.

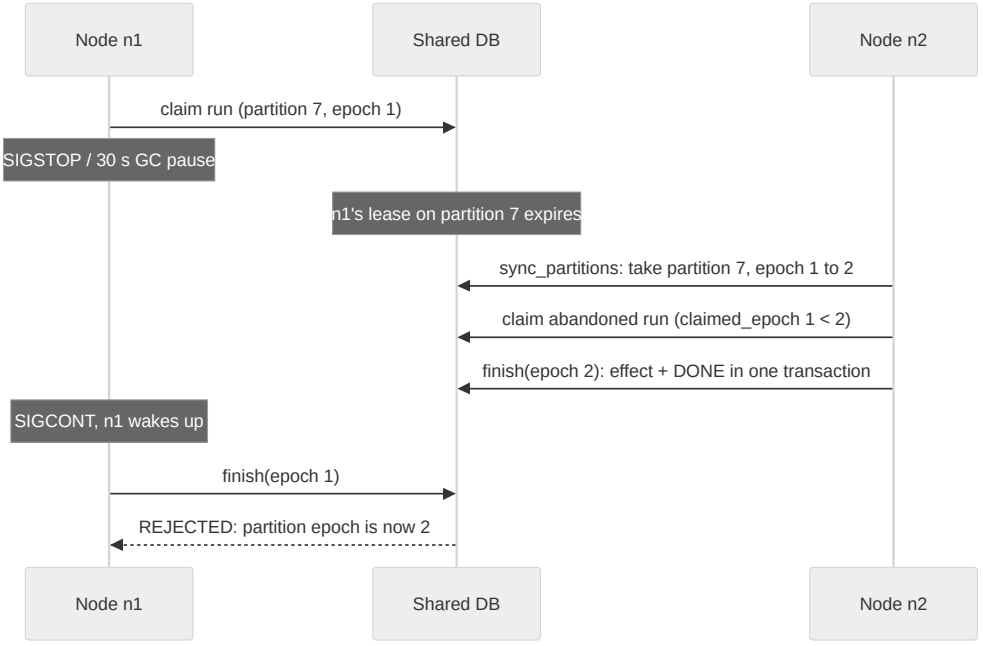
Misfire policy (a run more than 60 s late on its first attempt):

- `FIRE_NOW` runs every missed occurrence as soon as possible.
- `SKIP` drops the missed occurrences and continues at the next future time.
- `COALESCE` runs once now for all the missed occurrences, then continues at the next future time.

Per-tenant fairness. The claim query ranks due runs per tenant (`ROW_NUMBER() OVER (PARTITION BY tenant)`) and orders by that rank, so every tenant's 1st run comes before any tenant's 2nd run. Each tenant also gets at most `tenant_cap − already_running` slots per node. In the test, 500 runs from tenant *big* could not push out tenant *small*'s 5 runs.

Retries. Each retry waits `min(60, 1·2^(attempt−1))` seconds, randomised to 50–100% (jitter). After 5 attempts the run goes to the dead-letter list, where an operator can inspect it (`GET /dead-letters`) and replay it.

3. Hardest flow: a frozen node wakes up after its lease expired



4. Failure modes

What fails	What the user sees	How the system recovers
Node crash (`kill -9`)	Jobs in its partitions pause for at most one lease (10 s)	Lease expires → others claim its partitions (epoch +1) → abandoned RUNNING runs (claimed under the old epoch) are re-run → idempotency key keeps one effect

Node freeze / GC pause	Same as a crash	Same takeover; when the node wakes, its commit fails the fencing check and is logged as `stale_write_rejected`
Job handler fails (downstream down)	Run shows as retrying in `/metrics`	Exponential backoff with jitter; after 5 attempts → dead-letter list → operator replays
Run longer than `max_runtime`	Counted as a failure	Retried, then dead-lettered
Whole cluster down	Jobs are late	On restart each job follows its misfire policy
Node joins	Nothing	Others hand back extra partitions; in-flight runs there are re-run safely (fencing + idempotency)
Database busy	Small delay	30 s busy timeout; node loop logs the error and keeps going
Bad API request	422 with field names only (input values are not echoed back)	Rejected before anything is stored
Database file lost / disk full	Scheduling stops	**Not handled**: single point of failure (see trade-offs)

5. Benchmark (real 3-process cluster)

Run	Jobs	Window	Jobs/s	Done / effects	p50 lag	p99 lag	Max lag	Target p99 < 2 s
Brief's target	10,000	60 s	166.7	10,000 / 10,000	0.098 s	**0.199 s**	0.269 s	PASS
Short run, same rate	10,000	60 s	166.7	10,000 / 10,000	0.098 s	**0.199 s**	0.269 s	PASS

Machine: Linux-6.18.44-fc-v185-x86_64-with-glibc2.39 | 2 CPUs | Python 3.13.18. 20 tenants, 5 ms of simulated work per job, poll interval 200 ms (most of the lag is that poll interval). Command: `python bench/bench.py --jobs 100000 --window 600`.

6. Cost (time and space) and where it stops scaling

- `sync_partitions`: O(P) per node per renewal (P = 64).
- `claim_due`: index range scan on `(status, due_at)` plus a sort of the D due runs, so O(D log D) per poll.
- `finish`: O(1) indexed reads and writes. Next fire time: O(1) cron step (up to 60 steps across a DST gap).
- Memory per node: O(workers + batch). Storage: O(jobs + run history).

Where it stops scaling: SQLite has a single write lock, so all claims and commits across all nodes are serialised. 167 jobs/s used a small fraction of it (p99 0.2 s), but the limit was not pushed to breaking point. Each Python process uses about one core for Python code (GIL), so you add capacity by adding nodes. The next step is Postgres: the partition design already lets nodes write different rows in parallel.

7. Trade-offs and what was deliberately left out

- **SQLite instead of Postgres**, for a one-command run with no database server. The cost: one machine and one write lock. All database code is in `store.py`, so swapping it is contained.
- **Consistency over availability:** if the database is unreachable, nothing fires, rather than risk double effects.
- **Lease = 10 s:** shorter means faster takeover but more renewal writes and more false takeovers during short pauses.
- **Polling every 200 ms** instead of an in-memory timer heap: simpler, and it meets the target. The poll interval is the main source of lag.
- **Tenant cap is per node** (cluster-wide cap = cap × nodes), which avoids a cross-node counter.
- `max_runtime` is checked when the handler returns, because Python threads cannot be killed. A production version would run handlers in a subprocess.
- **Left out:** Docker, a Prometheus exporter (`/metrics` returns JSON computed from the shared database, so it covers the whole cluster), JWT/OAuth (static tokens that map to a role and a tenant), a real SMS provider (the `effects` table is the outbox), and editing or deleting jobs.

8. Tests

```
placeholder
22 passed in 25.0s
```

Every hard rule has a test:

- DST gap and fold
- rebalancing when a node joins or leaves
- a stale commit rejected
- retries ending in the dead-letter list, then replay
- the same feed twice creating one run
- fairness
- all 3 misfire policies
- pause and resume
- real-process `kill -9` (concurrency + failure) and `SIGSTOP`
- auth and roles
- tenant isolation
- validation errors that don't leak input

9. Run it

```
pip install -r requirements.txt
python run_cluster.py              # 3 nodes + API on http://127.0.0.1:8000  (docs at /docs)
python -m pytest -q                # all tests
python bench/bench.py --jobs 100000 --window 600

H='Authorization: Bearer acme-operator'
curl -X POST localhost:8000/jobs -H "$H" -H 'Content-Type: application/json' \
  -d '{"job_id":"roster-10pm","job_type":"send_alert","payload":{"employee_id":"E1"},"cron":"0 22
* * *","tz":"Asia/Kolkata"}'
curl "localhost:8000/jobs/roster-10pm/next?count=3" -H "$H"
curl -X POST localhost:8000/jobs/roster-10pm/trigger -H "$H"
curl -X POST localhost:8000/tenants/acme/pause -H "$H"
curl localhost:8000/dead-letters -H "$H"
curl localhost:8000/metrics -H "$H"
```

Commit history

```
0639302  Add job model, settings and time-zone/DST rules with tests
0999eba  Add SQLite store with partition leases and fencing, scheduler node, handlers and unit
tests
ff9bc2c  Add kill -9 and SIGSTOP freeze tests with real worker processes
0935d07  Add FastAPI control API with token roles and tenant isolation, one-command launcher,
benchmark
```

Appendix: source code

scheduler/config.py

```
"""All tunable settings in one place. Every value can be overridden by an env var."""
import os
from dataclasses import dataclass


def _env(name: str, default):
    raw = os.environ.get(name)
    return type(default)(raw) if raw is not None else default


@dataclass(frozen=True)
class Settings:
    db_path: str = _env("SCHED_DB", "scheduler.db")
    partitions: int = _env("SCHED_PARTITIONS", 64)          # fixed number of buckets
    lease_seconds: float = _env("SCHED_LEASE_SECONDS", 10.0)  # how long a node owns a bucket without
renewing
    poll_seconds: float = _env("SCHED_POLL_SECONDS", 0.2)     # how often a node looks for due runs
    workers: int = _env("SCHED_WORKERS", 32)                  # threads per node that execute runs
    tenant_cap: int = _env("SCHED_TENANT_CAP", 16)            # max concurrent runs per tenant per node
    batch_size: int = _env("SCHED_BATCH", 200)                # max runs fetched per poll
    max_attempts: int = _env("SCHED_MAX_ATTEMPTS", 5)         # then dead-letter
    backoff_base: float = _env("SCHED_BACKOFF_BASE", 1.0)     # seconds
    backoff_max: float = _env("SCHED_BACKOFF_MAX", 60.0)
    misfire_seconds: float = _env("SCHED_MISFIRE_SECONDS", 60.0)  # later than this = misfire


def load() -> Settings:
    return Settings()
```

scheduler/models.py

```
"""Plain data. No database or web code here."""
from dataclasses import dataclass, field
from enum import Enum
from typing import Optional


class MisfirePolicy(str, Enum):
    FIRE_NOW = "FIRE_NOW"   # run every missed occurrence as soon as possible
    SKIP = "SKIP"           # drop missed occurrences, continue at the next future time
    COALESCE = "COALESCE"   # run ONCE now for all missed occurrences, then continue


@dataclass
class Job:
    job_id: str
    tenant_id: str
    job_type: str                     # which handler runs it, e.g. "send_alert"
    payload: dict = field(default_factory=dict)
    run_at: Optional[float] = None    # one-off: unix seconds (UTC)
    cron: Optional[str] = None        # recurring: cron expression in the tenant's time zone
    tz: str = "UTC"
    priority: int = 0                 # higher runs first
    max_runtime: float = 30.0         # seconds; longer counts as a failure
    misfire: MisfirePolicy = MisfirePolicy.FIRE_NOW
```

scheduler/timeutil.py

```
"""Cron + time zones + daylight-saving rules. Pure functions, easy to test.

Rules (written down because the brief asks for them):
  * GAP  - a wall time that does not exist (02:30 when clocks jump 02:00 -> 03:00)
           runs at the moment the clock jumps (03:00 local).
  * FOLD - a wall time that happens twice (01:30 when clocks go 02:00 -> 01:00)
           runs ONCE, at the first occurrence.
```

```
"""
from datetime import datetime, timedelta, timezone
from zoneinfo import ZoneInfo

from croniter import croniter

UTC = timezone.utc


def local_to_utc(naive: datetime, tz: ZoneInfo) -> datetime:
    """Turn a wall-clock time in `tz` into a UTC instant, applying the GAP/FOLD rules."""
    utc = naive.replace(tzinfo=tz, fold=0).astimezone(UTC)  # fold=0 -> first occurrence
    if utc.astimezone(tz).replace(tzinfo=None) == naive:
        return utc  # the wall time exists (if it repeats, fold=0 already picked the first one)
    # GAP: fold=0 maps a missing time to after the jump (02:30 -> 03:30).
    # Walk back minute by minute to the instant the clock jumped (03:00).
    while (utc - timedelta(minutes=1)).astimezone(tz).replace(tzinfo=None) >= naive:
        utc -= timedelta(minutes=1)
    return utc


def next_fire(cron: str, tz_name: str, after: float) -> float:
    """First fire time strictly after `after` (unix seconds). Cron is read in the tenant's zone."""
    tz = ZoneInfo(tz_name)
    local_after = datetime.fromtimestamp(after, UTC).astimezone(tz).replace(tzinfo=None)
    it = croniter(cron, local_after)
    while True:
        candidate = local_to_utc(it.get_next(datetime), tz).timestamp()
        if candidate > after:   # guards the FOLD case: never fire the repeated hour twice
            return candidate


def next_fires(cron: str, tz_name: str, after: float, count: int) -> list[float]:
    out, t = [], after
    for _ in range(count):
        t = next_fire(cron, tz_name, t)
        out.append(t)
    return out


def validate_cron(cron: str) -> bool:
    return croniter.is_valid(cron)
```

scheduler/store.py

```
"""All database access (repository layer). SQLite file shared by every node.

  partitions : 64 buckets. A node owns a bucket through a timed lease.
  epoch      : +1 every time a bucket changes owner = the FENCING TOKEN.
  idem_key   : job_id + scheduled time. effects.idem_key is unique -> effect happens once.
  clock      : the database's clock decides due times and lease expiry (not each node's clock).
"""
import json
import math
import sqlite3
import threading
import uuid
import zlib
from contextlib import contextmanager
from typing import Optional

from .models import Job
from .timeutil import next_fire

SCHEMA = """
CREATE TABLE IF NOT EXISTS jobs (job_id TEXT PRIMARY KEY, tenant_id TEXT, job_type TEXT, payload TEXT,
    run_at REAL, cron TEXT, tz TEXT, priority INTEGER, max_runtime REAL, misfire TEXT);
CREATE TABLE IF NOT EXISTS runs (run_id INTEGER PRIMARY KEY AUTOINCREMENT, job_id TEXT, tenant_id TEXT,
    partition INTEGER, scheduled_at REAL, due_at REAL, status TEXT, attempts INTEGER DEFAULT 0,
    idem_key TEXT UNIQUE, claimed_by TEXT, claimed_epoch INTEGER, first_started_at REAL, last_error TEXT);
CREATE INDEX IF NOT EXISTS runs_status_due ON runs(status, due_at);
CREATE TABLE IF NOT EXISTS partitions (pid INTEGER PRIMARY KEY, owner TEXT, epoch INTEGER DEFAULT 0,
    lease_until REAL DEFAULT 0);
```

```
CREATE TABLE IF NOT EXISTS nodes (node_id TEXT PRIMARY KEY, heartbeat REAL);
CREATE TABLE IF NOT EXISTS tenants (tenant_id TEXT PRIMARY KEY, paused INTEGER DEFAULT 0);
CREATE TABLE IF NOT EXISTS effects (idem_key TEXT PRIMARY KEY, run_id INTEGER, tenant_id TEXT,
    kind TEXT, target TEXT, node_id TEXT);
"""
NOW = "(julianday('now') - 2440587.5) * 86400.0"   # unix seconds from the DB clock


class Store:
    def __init__(self, path: str, partitions: int = 64):
        self.path, self.partitions = path, partitions
        self._local = threading.local()   # sqlite connections are per thread

    @property
    def conn(self) -> sqlite3.Connection:
        if not hasattr(self._local, "c"):
            c = sqlite3.connect(self.path, timeout=30, isolation_level=None)
            c.row_factory = sqlite3.Row
            c.execute("PRAGMA journal_mode=WAL")
            self._local.c = c
        return self._local.c

    @contextmanager
    def tx(self):
        """BEGIN IMMEDIATE takes the write lock first, so read-check-write is atomic."""
        self.conn.execute("BEGIN IMMEDIATE")
        try:
            yield self.conn
            self.conn.execute("COMMIT")
        except BaseException:
            self.conn.execute("ROLLBACK")
            raise

    def init_schema(self):
        self.conn.executescript(SCHEMA)
        with self.tx() as c:
            c.executemany("INSERT OR IGNORE INTO partitions(pid) VALUES (?)", [(p,) for p in
range(self.partitions)])

    def now(self) -> float:
        return self.conn.execute(f"SELECT {NOW}").fetchone()[0]

    def partition_of(self, job_id: str) -> int:
        return zlib.crc32(job_id.encode()) % self.partitions   # stable across processes (hash() is not)

    def _add_run(self, c, job_id, tenant_id, at, key):
        c.execute("INSERT OR IGNORE INTO runs(job_id, tenant_id, partition, scheduled_at, due_at, status,
idem_key)"
                  " VALUES (?,?,?,?,?,'PENDING',?)", (job_id, tenant_id, self.partition_of(job_id), at, at,
key))

    # ---------------- jobs ----------------
    def create_jobs(self, jobs: list[Job]):
        with self.tx() as c:
            now = c.execute(f"SELECT {NOW}").fetchone()[0]
            for j in jobs:
                c.execute("INSERT INTO jobs VALUES (?,?,?,?,?,?,?,?,?,?)", (j.job_id, j.tenant_id,
j.job_type,
                          json.dumps(j.payload), j.run_at, j.cron, j.tz, j.priority, j.max_runtime,
j.misfire.value))
                c.execute("INSERT OR IGNORE INTO tenants(tenant_id) VALUES (?)", (j.tenant_id,))
                first = next_fire(j.cron, j.tz, now) if j.cron else j.run_at
                self._add_run(c, j.job_id, j.tenant_id, first, f"{j.job_id}:{first:.3f}")

    def get_job(self, job_id: str):
        return self.conn.execute("SELECT * FROM jobs WHERE job_id=?", (job_id,)).fetchone()

    # ---------------- leases + rebalancing ----------------
    def sync_partitions(self, node_id: str, lease: float) -> dict[int, int]:
        """Heartbeat, renew my leases, hand back extras, claim my fair share. Returns {pid: epoch}."""
        with self.tx() as c:
            now = c.execute(f"SELECT {NOW}").fetchone()[0]
            c.execute("INSERT OR REPLACE INTO nodes VALUES (?,?)", (node_id, now))
            live = c.execute("SELECT COUNT(*) FROM nodes WHERE heartbeat >= ?", (now - lease,)).fetchone()[0]
            share = math.ceil(self.partitions / max(live, 1))
```

```
            c.execute("UPDATE partitions SET lease_until=? WHERE owner=?", (now + lease, node_id))  # renew
(only if still mine)
            mine = [r[0] for r in c.execute("SELECT pid FROM partitions WHERE owner=? ORDER BY pid",
(node_id,))]
            if len(mine) > share:   # a node joined -> give back extras
                c.executemany("UPDATE partitions SET owner=NULL, lease_until=0 WHERE pid=?", [(p,) for p in
mine[share:]])
            elif len(mine) < share:  # a node died (lease expired) or I just started -> take free buckets
                free = [r[0] for r in c.execute("SELECT pid FROM partitions WHERE owner IS NULL OR
lease_until < ?"
                                                " ORDER BY pid LIMIT ?", (now, share - len(mine)))]
                c.executemany("UPDATE partitions SET owner=?, epoch=epoch+1, lease_until=? WHERE pid=?",
                              [(node_id, now + lease, p) for p in free])
            return {r[0]: r[1] for r in c.execute("SELECT pid, epoch FROM partitions WHERE owner=?",
(node_id,))}

    # ---------------- claim due runs ----------------
    def claim_due(self, node_id: str, limit: int, tenant_cap: int) -> list[dict]:
        """Claim due runs in my buckets and stamp each with the bucket's epoch (its fencing token).

        Due = PENDING and due_at <= now, OR RUNNING but claimed under an older epoch (its node died).
        Fairness: rank runs per tenant and take at most (cap - already running) per tenant, round-robin.
        """
        if limit <= 0:
            return []
        with self.tx() as c:
            now = c.execute(f"SELECT {NOW}").fetchone()[0]
            ids = [r[0] for r in c.execute("""
                WITH due AS (
                  SELECT r.run_id, r.tenant_id, j.priority, r.due_at FROM runs r
                    JOIN partitions p ON p.pid = r.partition JOIN jobs j ON j.job_id = r.job_id
                   WHERE p.owner = :me AND ((r.status = 'PENDING' AND r.due_at <= :now)
                                         OR (r.status = 'RUNNING' AND r.claimed_epoch < p.epoch))),
                ranked AS (
                  SELECT d.*, ROW_NUMBER() OVER (PARTITION BY d.tenant_id ORDER BY d.priority DESC,
d.due_at) rn,
                         (SELECT COUNT(*) FROM runs x WHERE x.status = 'RUNNING' AND x.claimed_by = :me
                             AND x.tenant_id = d.tenant_id) busy
                    FROM due d JOIN tenants t ON t.tenant_id = d.tenant_id WHERE t.paused = 0)
                SELECT run_id FROM ranked WHERE rn + busy <= :cap ORDER BY rn, priority DESC, due_at LIMIT
:limit""",
                {"me": node_id, "now": now, "cap": tenant_cap, "limit": limit})]
            if not ids:
                return []
            marks = ",".join("?" * len(ids))
            c.execute(f"""UPDATE runs SET status='RUNNING', claimed_by=?, attempts=attempts+1,
                              first_started_at=COALESCE(first_started_at, ?),
                              claimed_epoch=(SELECT epoch FROM partitions WHERE pid=runs.partition)
                        WHERE run_id IN ({marks})""", [node_id, now, *ids])
            rows = c.execute(f"""SELECT r.*, j.job_type, j.payload, j.max_runtime, j.misfire, j.cron, j.tz
                                   FROM runs r JOIN jobs j ON j.job_id = r.job_id WHERE r.run_id IN
({marks})""", ids)
            return [dict(r) | {"payload": json.loads(r["payload"])} for r in rows]

    # ---------------- the fenced write ----------------
    def finish(self, run: dict, node_id: str, status: str, effect=None, next_at=None, retry_at=None,
error=None) -> bool:
        """Save effect + new status + next occurrence in ONE transaction, only if my fencing token is
current.

        status: DONE | SKIPPED | DEAD | PENDING (retry at retry_at). Returns False if rejected as stale.
        """
        with self.tx() as c:
            epoch = c.execute("SELECT epoch FROM partitions WHERE pid=? AND owner=?",
                              (run["partition"], node_id)).fetchone()
            row = c.execute("SELECT status, claimed_by, claimed_epoch FROM runs WHERE run_id=?",
(run["run_id"],)).fetchone()
            if (epoch is None or epoch[0] != run["claimed_epoch"] or row["status"] != "RUNNING"
                    or row["claimed_by"] != node_id or row["claimed_epoch"] != run["claimed_epoch"]):
                return False   # stale: my lease expired and another node took this bucket
            if effect:
                c.execute("INSERT OR IGNORE INTO effects VALUES (?,?,?,?,?,?)",
                          (run["idem_key"], run["run_id"], run["tenant_id"], effect[0], effect[1], node_id))
            c.execute("UPDATE runs SET status=?, due_at=COALESCE(?, due_at), last_error=? WHERE run_id=?",
                      (status, retry_at, (error or "")[:200] or None, run["run_id"]))
```

```
            if next_at is not None:
                self._add_run(c, run["job_id"], run["tenant_id"], next_at, f"{run['job_id']}:{next_at:.3f}")
            return True

    # ---------------- operator controls ----------------
    def set_paused(self, tenant_id: str, paused: bool):
        with self.tx() as c:
            c.execute("INSERT OR REPLACE INTO tenants VALUES (?,?)", (tenant_id, int(paused)))

    def trigger_now(self, job_id: str, tenant_id: str):
        with self.tx() as c:
            self._add_run(c, job_id, tenant_id, c.execute(f"SELECT {NOW}").fetchone()[0], f"{job_id}:manual:
{uuid.uuid4().hex}")

    def dead_letters(self, tenant_id: Optional[str]) -> list[dict]:
        rows = self.conn.execute("SELECT run_id, job_id, tenant_id, attempts, last_error FROM runs WHERE
status='DEAD'"
                                 " AND (? IS NULL OR tenant_id=?) ORDER BY run_id", (tenant_id, tenant_id))
        return [dict(r) for r in rows]

    def replay(self, run_id: int, tenant_id: Optional[str]) -> bool:
        with self.tx() as c:
            return c.execute(f"UPDATE runs SET status='PENDING', attempts=0, due_at={NOW}, last_error=NULL"
                             " WHERE run_id=? AND status='DEAD' AND (? IS NULL OR tenant_id=?)",
                             (run_id, tenant_id, tenant_id)).rowcount == 1

    def metrics(self, tenant_id: Optional[str] = None) -> dict:
        """Fire lag histogram + running/retrying/dead counts, for the whole cluster."""
        f = "(? IS NULL OR tenant_id=?)"
        a = (tenant_id, tenant_id)
        q = lambda sql: self.conn.execute(sql, a).fetchone()[0]
        lags = sorted(r[0] for r in self.conn.execute(
            f"SELECT first_started_at - scheduled_at FROM runs WHERE first_started_at IS NOT NULL AND {f}",
a))
        buckets = [0.1, 0.25, 0.5, 1, 2, 5, 10, 60]
        pct = lambda p: round(lags[min(len(lags) - 1, int(p * len(lags)))], 3) if lags else None
        return {
            "running": q(f"SELECT COUNT(*) FROM runs WHERE status='RUNNING' AND {f}"),
            "retrying": q(f"SELECT COUNT(*) FROM runs WHERE status='PENDING' AND attempts > 0 AND {f}"),
            "dead_lettered": q(f"SELECT COUNT(*) FROM runs WHERE status='DEAD' AND {f}"),
            "done": q(f"SELECT COUNT(*) FROM runs WHERE status='DONE' AND {f}"),
            "fire_lag_histogram": {f"le_{b}s": sum(1 for x in lags if x <= b) for b in buckets} | {"total":
len(lags)},
            "fire_lag_p50_s": pct(0.50), "fire_lag_p99_s": pct(0.99),
        }
```

scheduler/handlers.py

```
"""Job types. A new type = a new subclass of JobHandler (polymorphism)."""
import time
from abc import ABC, abstractmethod
from typing import Optional


class JobHandler(ABC):
    @abstractmethod
    def execute(self, payload: dict) -> Optional[tuple[str, str]]:
        """Do the work. Return the effect to record as (kind, target), or None."""


class SendAlert(JobHandler):
    """'Your cab is assigned' alert. The effect row is the outbox a real SMS sender would read,
    so it is written in the same transaction as the run's status (see Store.finish)."""

    def execute(self, payload):
        time.sleep(payload.get("work_ms", 0) / 1000)   # simulated work
        return ("alert", str(payload["employee_id"]))   # employee ID only, never phone/email


class AlwaysFail(JobHandler):
    """Fails every time. Used to show retries and the dead-letter list."""

    def execute(self, payload):
```

```
        raise RuntimeError("downstream unavailable")


HANDLERS: dict[str, JobHandler] = {"send_alert": SendAlert(), "fail": AlwaysFail()}
```

scheduler/node.py

```
"""One scheduler node: keep bucket leases, claim due runs, execute them, save results (fenced)."""
import json
import logging
import random
import threading
import time
from concurrent.futures import ThreadPoolExecutor

from .config import Settings
from .handlers import HANDLERS
from .store import Store
from .timeutil import next_fire

log = logging.getLogger("scheduler")


def backoff(attempt: int, base: float, cap: float) -> float:
    """Exponential backoff with jitter: ~1s, 2s, 4s ... capped; each randomised to 50-100%
    so retries from many runs don't all hit the downstream at the same instant."""
    d = min(cap, base * 2 ** (attempt - 1))
    return d / 2 + random.uniform(0, d / 2)


class SchedulerNode:
    def __init__(self, node_id: str, store: Store, settings: Settings, handlers=HANDLERS):
        self.node_id, self.store, self.s, self.handlers = node_id, store, settings, handlers
        self.owned: dict[int, int] = {}
        self.pool = ThreadPoolExecutor(settings.workers)
        self.in_flight = 0
        self.lock = threading.Lock()
        self.stop_event = threading.Event()

    # ---- main loop ----
    def run_forever(self):
        next_sync = 0.0
        while not self.stop_event.is_set():
            try:
                if time.monotonic() >= next_sync:   # renew leases 3x per lease period
                    self.owned = self.store.sync_partitions(self.node_id, self.s.lease_seconds)
                    next_sync = time.monotonic() + self.s.lease_seconds / 3
                claimed = self.tick()
            except Exception as e:                  # e.g. DB busy: log, keep going
                log.error(json.dumps({"event": "loop_error", "node": self.node_id, "error":
type(e).__name__}))
                claimed = 0
            if not claimed:
                self.stop_event.wait(self.s.poll_seconds)

    def tick(self, inline: bool = False) -> int:
        """Claim due runs (up to free worker slots) and run them. inline=True runs them here (for tests)."""
        with self.lock:
            free = self.s.workers - self.in_flight
        runs = self.store.claim_due(self.node_id, min(free, self.s.batch_size), self.s.tenant_cap)
        for run in runs:
            if inline:
                self.execute(run)
                continue
            with self.lock:
                self.in_flight += 1
            self.pool.submit(self._safe_execute, run)
        return len(runs)

    def _safe_execute(self, run):
        try:
            self.execute(run)
        except Exception as e:
```

```
            log.error(json.dumps({"event": "execute_error", "run_id": run["run_id"], "error":
type(e).__name__}))
        finally:
            with self.lock:
                self.in_flight -= 1

    # ---- one run ----
    def execute(self, run: dict) -> str:
        now = self.store.now()
        cron, tz, policy = run["cron"], run["tz"], run["misfire"]
        next_after = lambda t: next_fire(cron, tz, t) if cron else None
        late = run["attempts"] == 1 and now - run["scheduled_at"] > self.s.misfire_seconds

        if late and policy == "SKIP":
            return self._save(run, "SKIPPED", next_at=next_after(now))
        # COALESCE: one run now stands in for all missed ones, so jump to the next future time.
        # FIRE_NOW: continue from the scheduled time, so every missed occurrence also runs.
        nxt = next_after(now if (late and policy == "COALESCE") else run["scheduled_at"])

        try:
            started = time.monotonic()
            effect = self.handlers[run["job_type"]].execute(run["payload"])
            if time.monotonic() - started > run["max_runtime"]:
                raise TimeoutError("exceeded max_runtime")
        except Exception as e:
            err = f"{type(e).__name__}: {e}"
            if run["attempts"] >= self.s.max_attempts:
                return self._save(run, "DEAD", next_at=nxt, error=err)
            retry_at = self.store.now() + backoff(run["attempts"], self.s.backoff_base, self.s.backoff_max)
            return self._save(run, "PENDING", retry_at=retry_at, error=err)
        return self._save(run, "DONE", effect=effect, next_at=nxt)

    def _save(self, run: dict, status: str, **kw) -> str:
        ok = self.store.finish(run, self.node_id, status, **kw)
        log.info(json.dumps({"event": "run_saved" if ok else "stale_write_rejected", "node": self.node_id,
                             "run_id": run["run_id"], "status": status, "epoch": run["claimed_epoch"]}))
        return status if ok else "REJECTED"
```

scheduler/worker.py

```
"""Start one scheduler node:   python -m scheduler.worker --node n1"""
import argparse
import logging
import signal
import sys

from .config import load
from .node import SchedulerNode
from .store import Store


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--node", required=True)
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(message)s", stream=sys.stdout)   # JSON lines

    settings = load()
    store = Store(settings.db_path, settings.partitions)
    store.init_schema()
    node = SchedulerNode(args.node, store, settings)
    signal.signal(signal.SIGTERM, lambda *_: node.stop_event.set())
    node.run_forever()


if __name__ == "__main__":
    main()
```

scheduler/api.py

```
"""Control API (FastAPI). Run:  uvicorn scheduler.api:create_app --factory
```

```
Login: 'Authorization: Bearer <token>'. Each token maps to a role and a tenant.
  admin    - every tenant
  operator - own tenant: create jobs, pause/resume, trigger, replay dead letters
  employee - own tenant: read only
Another tenant's job answers 404 (not 403), so you can't even learn that it exists.
"""
import json
import os
import sqlite3
from datetime import datetime, timezone
from typing import Optional
from zoneinfo import ZoneInfo

from fastapi import Depends, FastAPI, Header, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field

from .config import load
from .handlers import HANDLERS
from .models import Job, MisfirePolicy
from .store import Store
from .timeutil import next_fires, validate_cron

DEMO_TOKENS = {"admin-token": ["admin", None], "acme-operator": ["operator", "acme"],
               "acme-employee": ["employee", "acme"], "globex-operator": ["operator", "globex"]}


class ApiError(Exception):
    def __init__(self, status: int, code: str, message: str):
        self.status, self.code, self.message = status, code, message


class User(BaseModel):
    role: str
    tenant: Optional[str]


class JobIn(BaseModel):
    job_id: str = Field(min_length=1, max_length=100)
    tenant_id: Optional[str] = None          # only admins may set it; others use their own tenant
    job_type: str
    payload: dict = {}
    run_at: Optional[float] = None           # one-off: unix seconds
    cron: Optional[str] = None               # recurring
    tz: str = "UTC"
    priority: int = 0
    max_runtime: float = Field(30, gt=0)
    misfire: MisfirePolicy = MisfirePolicy.FIRE_NOW


def create_app(store: Optional[Store] = None) -> FastAPI:
    settings = load()
    store = store or Store(settings.db_path, settings.partitions)
    store.init_schema()
    tokens = json.loads(os.environ.get("SCHED_TOKENS", "null")) or DEMO_TOKENS
    app = FastAPI(title="Trip Job Scheduler")

    # ---------- errors: clear message, no internals, no personal data ----------
    @app.exception_handler(ApiError)
    async def api_error(_: Request, e: ApiError):
        return JSONResponse({"error": e.code, "message": e.message}, status_code=e.status)

    @app.exception_handler(RequestValidationError)
    async def bad_request(_: Request, e: RequestValidationError):   # field names only, never the input
values
        fields = [".".join(str(p) for p in err["loc"][1:]) for err in e.errors()]
        return JSONResponse({"error": "invalid_request", "message": f"Invalid fields: {fields}"},
status_code=422)

    @app.exception_handler(Exception)
    async def unexpected(_: Request, e: Exception):
        return JSONResponse({"error": "internal", "message": "Something went wrong"}, status_code=500)

    # ---------- auth ----------
```

```
    def user(authorization: str = Header(default="")) -> User:
        role_tenant = tokens.get(authorization.removeprefix("Bearer ").strip())
        if not role_tenant:
            raise ApiError(401, "unauthorized", "Missing or invalid token")
        return User(role=role_tenant[0], tenant=role_tenant[1])

    def writer(u: User = Depends(user)) -> User:
        if u.role == "employee":
            raise ApiError(403, "forbidden", "This role is read-only")
        return u

    def scope(u: User, tenant: Optional[str]) -> Optional[str]:
        """Which tenant may this user touch? Admin: whatever they ask (None = all)."""
        if u.role == "admin":
            return tenant
        if tenant and tenant != u.tenant:
            raise ApiError(404, "not_found", "Not found")
        return u.tenant

    def own_job(u: User, job_id: str):
        job = store.get_job(job_id)
        if not job or (u.role != "admin" and job["tenant_id"] != u.tenant):
            raise ApiError(404, "not_found", "Job not found")
        return job

    # ---------- endpoints ----------
    @app.get("/health")
    def health():
        return {"status": "ok"}

    @app.post("/jobs", status_code=201)
    def create_job(body: JobIn, u: User = Depends(writer)):
        tenant = scope(u, body.tenant_id)
        if not tenant:
            raise ApiError(422, "invalid_request", "tenant_id is required for admins")
        if (body.run_at is None) == (body.cron is None):
            raise ApiError(422, "invalid_request", "Give exactly one of run_at (one-off) or cron
(recurring)")
        if body.cron and not validate_cron(body.cron):
            raise ApiError(422, "invalid_request", "Invalid cron expression")
        if body.job_type not in HANDLERS:
            raise ApiError(422, "invalid_request", f"Unknown job_type; use one of {sorted(HANDLERS)}")
        if body.job_type == "send_alert" and "employee_id" not in body.payload:
            raise ApiError(422, "invalid_request", "send_alert needs payload.employee_id")
        try:
            ZoneInfo(body.tz)
        except Exception:
            raise ApiError(422, "invalid_request", "Unknown time zone")
        try:
            store.create_jobs([Job(**(body.model_dump() | {"tenant_id": tenant}))])
        except sqlite3.IntegrityError:
            raise ApiError(409, "conflict", "job_id already exists")
        return {"job_id": body.job_id, "tenant_id": tenant}

    @app.get("/jobs/{job_id}/next")
    def next_fire_times(job_id: str, count: int = 5, u: User = Depends(user)):
        job = own_job(u, job_id)
        if not job["cron"]:
            times = [job["run_at"]]
        else:
            times = next_fires(job["cron"], job["tz"], store.now(), min(count, 50))
        tz = ZoneInfo(job["tz"])
        return {"job_id": job_id, "tz": job["tz"], "next": [
            {"utc": datetime.fromtimestamp(t, timezone.utc).isoformat(),
             "local": datetime.fromtimestamp(t, tz).isoformat()} for t in times]}

    @app.post("/jobs/{job_id}/trigger", status_code=202)
    def trigger(job_id: str, u: User = Depends(writer)):
        job = own_job(u, job_id)
        store.trigger_now(job_id, job["tenant_id"])
        return {"job_id": job_id, "triggered": True}

    @app.post("/tenants/{tenant_id}/pause")
    def pause(tenant_id: str, u: User = Depends(writer)):
        store.set_paused(scope(u, tenant_id), True)
```

```
        return {"tenant_id": tenant_id, "paused": True}

    @app.post("/tenants/{tenant_id}/resume")
    def resume(tenant_id: str, u: User = Depends(writer)):
        store.set_paused(scope(u, tenant_id), False)
        return {"tenant_id": tenant_id, "paused": False}

    @app.get("/dead-letters")
    def dead_letters(u: User = Depends(user)):
        return store.dead_letters(scope(u, None))

    @app.post("/dead-letters/{run_id}/replay")
    def replay(run_id: int, u: User = Depends(writer)):
        if not store.replay(run_id, scope(u, None)):
            raise ApiError(404, "not_found", "Dead-lettered run not found")
        return {"run_id": run_id, "replayed": True}

    @app.get("/metrics")
    def metrics(u: User = Depends(user)):
        return store.metrics(scope(u, None))

    return app
```

run_cluster.py

```
"""One command to run everything:   python run_cluster.py

Starts 3 scheduler nodes + the API at http://127.0.0.1:8000 (interactive docs at /docs).
All processes share one SQLite file (SCHED_DB, default scheduler.db). Ctrl+C stops everything.
"""
import os
import subprocess
import sys
import time

from scheduler.config import load
from scheduler.store import Store

NODES = int(os.environ.get("NODES", 3))

if __name__ == "__main__":
    s = load()
    Store(s.db_path, s.partitions).init_schema()   # create tables once, before anyone starts
    procs = [subprocess.Popen([sys.executable, "-m", "scheduler.worker", "--node", f"n{i}"]) for i in
range(1, NODES + 1)]
    procs.append(subprocess.Popen([sys.executable, "-m", "uvicorn", "scheduler.api:create_app", "--factory",
                                   "--port", os.environ.get("PORT", "8000"), "--log-level", "warning"]))
    print(f"{NODES} nodes + API running on http://127.0.0.1:8000  (docs: /docs, token: acme-operator). Ctrl+C
to stop.")
    try:
        while all(p.poll() is None for p in procs):
            time.sleep(0.5)
    except KeyboardInterrupt:
        pass
    finally:
        for p in procs:
            p.terminate()
        for p in procs:
            p.wait()
```

bench/bench.py

```
"""Fire-lag benchmark on a real 3-process cluster.

  python bench/bench.py --jobs 100000 --window 600   # the brief: 100k jobs due within 10 minutes
  python bench/bench.py --jobs 10000 --window 60     # same rate (~167 jobs/s), 1 minute

Fire lag = time a run actually started - time it was scheduled. Target: p99 < 2 s.
"""
import argparse
import json
import os
```

```
import platform
import subprocess
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from scheduler.models import Job   # noqa: E402
from scheduler.store import Store  # noqa: E402


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--jobs", type=int, default=10000)
    ap.add_argument("--window", type=float, default=60)
    ap.add_argument("--nodes", type=int, default=3)
    ap.add_argument("--tenants", type=int, default=20)
    ap.add_argument("--work-ms", type=int, default=5)
    ap.add_argument("--db", default=str(ROOT / "bench" / "bench.db"))
    a = ap.parse_args()

    for suffix in ("", "-wal", "-shm"):
        Path(a.db + suffix).unlink(missing_ok=True)
    store = Store(a.db)
    store.init_schema()
    env = os.environ | {"SCHED_DB": a.db}
    procs = [subprocess.Popen([sys.executable, "-m", "scheduler.worker", "--node", f"n{i}"], cwd=ROOT,
env=env,
                              stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL) for i in range(1, a.nodes
+ 1)]
    try:
        q = lambda sql: store.conn.execute(sql).fetchone()[0]
        while q("SELECT COUNT(DISTINCT owner) FROM partitions") < a.nodes:   # wait for buckets to spread
            time.sleep(0.2)

        start = store.now() + 10
        store.create_jobs([Job(f"job-{i}", f"tenant{i % a.tenants}", "send_alert",
                               {"employee_id": f"E{i}", "work_ms": a.work_ms},
                               run_at=start + a.window * i / a.jobs) for i in range(a.jobs)])
        print(f"inserted {a.jobs} jobs due over {a.window:.0f}s on {a.nodes} nodes", flush=True)

        deadline = time.time() + a.window + 180
        while time.time() < deadline:
            done = q("SELECT COUNT(*) FROM runs WHERE status='DONE'")
            if done >= a.jobs:
                break
            print(f"  done {done}/{a.jobs}", flush=True)
            time.sleep(10)

        m = store.metrics()
        result = {
            "jobs": a.jobs, "window_s": a.window, "nodes": a.nodes, "tenants": a.tenants, "work_ms":
a.work_ms,
            "done": m["done"], "effects": q("SELECT COUNT(*) FROM effects"),
            "fire_lag_p50_s": m["fire_lag_p50_s"], "fire_lag_p99_s": m["fire_lag_p99_s"],
            "fire_lag_max_s": round(q("SELECT MAX(first_started_at - scheduled_at) FROM runs"), 3),
            "throughput_jobs_per_s": round(a.jobs / a.window, 1),
            "machine": f"{platform.platform()} | {os.cpu_count()} CPUs | Python {platform.python_version()}",
        }
        print(json.dumps(result, indent=2))
        out = ROOT / "bench" / f"results_{a.jobs}.json"
        out.write_text(json.dumps(result, indent=2))
    finally:
        for p in procs:
            p.terminate()
            p.wait()


if __name__ == "__main__":
    main()
```

tests/test_timeutil.py

```
"""Daylight-saving and time-zone rules. US clocks in 2026: jump forward 8 Mar, fall back 1 Nov."""
from datetime import datetime

from scheduler.timeutil import next_fire, next_fires, UTC


def ts(*args) -> float:
    return datetime(*args, tzinfo=UTC).timestamp()


def iso(t: float) -> str:
    return datetime.fromtimestamp(t, UTC).strftime("%Y-%m-%d %H:%M")


def test_kolkata_10pm_roster_job():
    # 22:00 IST = 16:30 UTC (India has no DST)
    assert iso(next_fire("0 22 * * *", "Asia/Kolkata", ts(2026, 10, 31, 0, 0))) == "2026-10-31 16:30"


def test_gap_0230_runs_when_clock_jumps_to_0300():
    # New York, 8 Mar 2026: 02:00 EST -> 03:00 EDT. 02:30 does not exist.
    t = next_fire("30 2 * * *", "America/New_York", ts(2026, 3, 7, 12, 0))
    assert iso(t) == "2026-03-08 07:00"   # 07:00 UTC = 03:00 EDT
    # and the day after is back to normal 02:30 EDT = 06:30 UTC
    assert iso(next_fire("30 2 * * *", "America/New_York", t)) == "2026-03-09 06:30"


def test_fold_0130_runs_once_at_first_occurrence():
    # New York, 1 Nov 2026: 02:00 EDT -> 01:00 EST. 01:30 happens twice.
    fires = next_fires("30 1 * * *", "America/New_York", ts(2026, 10, 31, 12, 0), 2)
    assert iso(fires[0]) == "2026-11-01 05:30"   # first 01:30 (EDT, UTC-4)
    assert iso(fires[1]) == "2026-11-02 06:30"   # next day, 01:30 EST (UTC-5) - not 06:30 on 1 Nov


def test_hourly_job_does_not_repeat_the_folded_hour():
    fires = next_fires("0 * * * *", "America/New_York", ts(2026, 11, 1, 4, 30), 3)
    # 01:00 EDT (05:00 UTC) runs, the second 01:00 (EST, 06:00 UTC) is skipped, then 02:00 EST
    assert [iso(f) for f in fires] == ["2026-11-01 05:00", "2026-11-01 07:00", "2026-11-01 08:00"]
```

tests/test_node.py

```
"""Core guarantees, tested in one process with no threads (tick(inline=True))."""
import pytest

from scheduler.config import Settings
from scheduler.models import Job, MisfirePolicy
from scheduler.node import SchedulerNode
from scheduler.store import Store


@pytest.fixture
def store(tmp_path):
    s = Store(str(tmp_path / "t.db"), partitions=8)
    s.init_schema()
    return s


def make_node(store, name, **kw):
    settings = Settings(**({"partitions": 8, "lease_seconds": 5, "backoff_base": 0.01, "backoff_max": 0.02} |
kw))
    node = SchedulerNode(name, store, settings)
    node.owned = store.sync_partitions(name, settings.lease_seconds)
    return node


def alert(job_id, tenant="acme", at=None, **kw):
    return Job(job_id, tenant, "send_alert", {"employee_id": job_id}, run_at=at, **kw)


def q(store, sql, *args):
    return store.conn.execute(sql, args).fetchone()[0]
```

```
def expire(store, node_id):
    """Pretend node_id froze: its heartbeat and leases are in the past."""
    store.conn.execute("UPDATE nodes SET heartbeat = 0 WHERE node_id = ?", (node_id,))
    store.conn.execute("UPDATE partitions SET lease_until = 0 WHERE owner = ?", (node_id,))


def test_rebalance_on_join_and_leave(store):
    a = make_node(store, "a")
    assert len(a.owned) == 8                        # alone -> owns every bucket
    b = make_node(store, "b")
    a.owned = store.sync_partitions("a", 5)         # b joined -> a hands back half
    b.owned = store.sync_partitions("b", 5)
    assert len(a.owned) == 4 and len(b.owned) == 4
    expire(store, "a")                              # a dies
    b.owned = store.sync_partitions("b", 5)
    assert len(b.owned) == 8                        # b takes over everything


def test_frozen_node_cannot_commit_after_lease_expired(store):
    store.create_jobs([alert("j1", at=store.now() - 1)])
    a = make_node(store, "a")
    run = store.claim_due("a", 10, 10)[0]           # a claims the run ... then freezes
    expire(store, "a")
    b = make_node(store, "b")                        # b takes the bucket: epoch goes up
    assert b.tick(inline=True) == 1                  # b re-runs the abandoned run and commits
    assert a.execute(run) == "REJECTED"              # a wakes up: stale fencing token -> rejected
    assert q(store, "SELECT COUNT(*) FROM effects") == 1
    assert q(store, "SELECT node_id FROM effects") == "b"


def test_retries_with_backoff_then_dead_letter_then_replay(store):
    store.create_jobs([Job("bad", "acme", "fail", run_at=store.now() - 1)])
    node = make_node(store, "a", max_attempts=3)
    import time
    for _ in range(100):
        node.tick(inline=True)
        if q(store, "SELECT status FROM runs") == "DEAD":
            break
        time.sleep(0.02)
    assert q(store, "SELECT attempts FROM runs") == 3
    dead = store.dead_letters("acme")
    assert len(dead) == 1 and "downstream unavailable" in dead[0]["last_error"]
    assert store.replay(dead[0]["run_id"], "acme")
    assert q(store, "SELECT status FROM runs") == "PENDING"


def test_same_feed_twice_creates_one_run(store):
    j = alert("j1", at=store.now() + 60)
    store.create_jobs([j])
    store._add_run(store.conn, "j1", "acme", j.run_at, f"j1:{j.run_at:.3f}")   # same idempotency key again
    assert q(store, "SELECT COUNT(*) FROM runs") == 1


def test_big_tenant_does_not_starve_small_tenant(store):
    now = store.now() - 1
    store.create_jobs([alert(f"big{i}", "big", now) for i in range(500)] +
                      [alert(f"small{i}", "small", now) for i in range(5)])
    make_node(store, "a")
    claimed = store.claim_due("a", limit=20, tenant_cap=10)
    tenants = [r["tenant_id"] for r in claimed]
    assert tenants.count("small") == 5               # all 20 small-tenant slots go first
    assert tenants.count("big") == 10                # big tenant is capped at 10


def past_recurring(store, policy):
    store.create_jobs([Job("cron1", "acme", "send_alert", {"employee_id": "e1"}, cron="* * * * *",
misfire=policy)])
    old = store.now() - 600                          # pretend the cluster was down for 10 minutes
    store.conn.execute("UPDATE runs SET scheduled_at=?, due_at=?, idem_key=?", (old, old, f"cron1:
{old:.3f}"))
    return old
```

```
@pytest.mark.parametrize("policy", list(MisfirePolicy))
def test_misfire_policies(store, policy):
    old = past_recurring(store, policy)
    node = make_node(store, "a")
    node.tick(inline=True)
    nxt = store.conn.execute("SELECT scheduled_at FROM runs WHERE status='PENDING'").fetchone()[0]
    effects = q(store, "SELECT COUNT(*) FROM effects")
    if policy is MisfirePolicy.SKIP:
        assert effects == 0 and nxt > store.now()                  # skipped, resumes in the future
    elif policy is MisfirePolicy.COALESCE:
        assert effects == 1 and nxt > store.now()                  # ran once, jumped to the future
    else:
        assert effects == 1 and abs(nxt - (old + 60)) < 61         # FIRE_NOW: next missed one is due too


def test_pause_and_resume_tenant(store):
    store.create_jobs([alert("j1", at=store.now() - 1)])
    node = make_node(store, "a")
    store.set_paused("acme", True)
    assert node.tick(inline=True) == 0
    store.set_paused("acme", False)
    assert node.tick(inline=True) == 1
```

tests/test_cluster.py

```
"""The two tests the brief asks for, with REAL processes and REAL signals.

  1. kill -9 a node while 1,200 pickup alerts are being sent -> every employee gets exactly one.
  2. SIGSTOP (freeze) a node past its lease -> another node finishes the job, and when the
     frozen node wakes up its stale write is rejected by the fencing token.
"""
import os
import signal
import subprocess
import sys
import time
from pathlib import Path

import pytest

from scheduler.models import Job
from scheduler.store import Store

ROOT = Path(__file__).resolve().parent.parent
pytestmark = pytest.mark.skipif(sys.platform == "win32", reason="needs POSIX signals")


def start(node, db, logdir, **env):
    e = os.environ | {"SCHED_DB": db, "SCHED_LEASE_SECONDS": "2", "SCHED_POLL_SECONDS": "0.1",
                      "SCHED_WORKERS": "8"} | {k: str(v) for k, v in env.items()}
    log = open(Path(logdir) / f"{node}.log", "w")
    return subprocess.Popen([sys.executable, "-m", "scheduler.worker", "--node", node],
                            cwd=ROOT, env=e, stdout=log, stderr=subprocess.STDOUT)


def wait_for(cond, timeout, step=0.1):
    end = time.time() + timeout
    while time.time() < end:
        if cond():
            return True
        time.sleep(step)
    return False


def q(store, sql, *args):
    return store.conn.execute(sql, args).fetchone()[0]


@pytest.fixture
def cluster(tmp_path):
    db = str(tmp_path / "c.db")
    store = Store(db)
    store.init_schema()
```

```
    procs = []
    yield store, db, tmp_path, procs
    for p in procs:
        if p.poll() is None:
            p.send_signal(signal.SIGCONT)
            p.kill()
            p.wait()


def test_kill_9_mid_batch_every_employee_gets_exactly_one_alert(cluster):
    store, db, logs, procs = cluster
    procs += [start(n, db, logs) for n in ("n1", "n2", "n3")]
    assert wait_for(lambda: q(store, "SELECT COUNT(DISTINCT owner) FROM partitions") == 3, 15)

    now = store.now()
    store.create_jobs([Job(f"pickup-{i}", f"tenant{i % 4}", "send_alert",
                           {"employee_id": f"E{i}", "work_ms": 200}, run_at=now + 0.5) for i in
range(1200)])
    assert wait_for(lambda: q(store, "SELECT COUNT(*) FROM effects") >= 200, 30)   # mid-batch...
    procs[0].send_signal(signal.SIGKILL)                                            # ... kill -9 n1

    assert wait_for(lambda: q(store, "SELECT COUNT(*) FROM runs WHERE status='DONE'") == 1200, 90)
    assert q(store, "SELECT COUNT(*) FROM effects") == 1200                  # exactly one alert each
    assert q(store, "SELECT COUNT(DISTINCT target) FROM effects") == 1200
    assert q(store, "SELECT COUNT(DISTINCT owner) FROM partitions") == 2     # n1's buckets moved
    assert q(store, "SELECT COUNT(*) FROM runs WHERE attempts >= 2") > 0     # some runs executed twice...
    # ...but their effect still happened once: at-least-once execution + idempotency = exactly-once effect.


def test_frozen_node_stale_write_is_rejected(cluster):
    store, db, logs, procs = cluster
    procs.append(start("n1", db, logs))
    assert wait_for(lambda: q(store, "SELECT COUNT(*) FROM partitions WHERE owner='n1'") == 64, 15)

    store.create_jobs([Job("close-billing", "acme", "send_alert",
                           {"employee_id": "E1", "work_ms": 3000}, run_at=store.now())])
    assert wait_for(lambda: q(store, "SELECT status FROM runs") == "RUNNING", 10)
    procs[0].send_signal(signal.SIGSTOP)                        # freeze n1 mid-job (like a long GC pause)

    procs.append(start("n2", db, logs))                         # n2 takes the expired buckets and re-runs it
    assert wait_for(lambda: q(store, "SELECT COUNT(*) FROM effects") == 1, 20)
    assert q(store, "SELECT node_id FROM effects") == "n2"

    procs[0].send_signal(signal.SIGCONT)                        # n1 wakes up and tries to save its result
    n1_log = logs / "n1.log"
    assert wait_for(lambda: "stale_write_rejected" in n1_log.read_text(), 15)
    assert q(store, "SELECT COUNT(*) FROM effects") == 1       # still exactly one effect, written by n2
```

tests/test_api.py

```
"""API: login, roles, tenant isolation, validation, operator controls."""
import pytest
from fastapi.testclient import TestClient

from scheduler.api import create_app
from scheduler.store import Store

ACME = {"Authorization": "Bearer acme-operator"}
ACME_EMPLOYEE = {"Authorization": "Bearer acme-employee"}
GLOBEX = {"Authorization": "Bearer globex-operator"}


@pytest.fixture
def client(tmp_path):
    return TestClient(create_app(Store(str(tmp_path / "api.db"))))


def roster_job(job_id="roster"):
    return {"job_id": job_id, "job_type": "send_alert", "payload": {"employee_id": "E1"},
            "cron": "0 22 * * *", "tz": "Asia/Kolkata"}


def test_needs_a_valid_token(client):
```

```
    assert client.get("/metrics").status_code == 401
    assert client.get("/metrics", headers={"Authorization": "Bearer nope"}).status_code == 401


def test_create_job_and_list_next_fire_times(client):
    assert client.post("/jobs", json=roster_job(), headers=ACME).status_code == 201
    r = client.get("/jobs/roster/next?count=3", headers=ACME).json()
    assert len(r["next"]) == 3 and all(t["local"].endswith("22:00:00+05:30") for t in r["next"])


def test_tenant_isolation_other_tenant_gets_404(client):
    client.post("/jobs", json=roster_job(), headers=ACME)
    assert client.get("/jobs/roster/next", headers=GLOBEX).status_code == 404
    assert client.post("/jobs/roster/trigger", headers=GLOBEX).status_code == 404
    assert client.post("/tenants/acme/pause", headers=GLOBEX).status_code == 404


def test_employee_is_read_only(client):
    assert client.post("/jobs", json=roster_job(), headers=ACME_EMPLOYEE).status_code == 403


def test_validation_errors_do_not_echo_input(client):
    bad = roster_job() | {"cron": "not a cron"}
    r = client.post("/jobs", json=bad, headers=ACME)
    assert r.status_code == 422 and "not a cron" not in r.text
    r = client.post("/jobs", json={"job_id": "x", "job_type": "send_alert", "payload": {"phone":
"+919999999999"},
                                   "max_runtime": -1}, headers=ACME)
    assert r.status_code == 422 and "9999" not in r.text
    no_employee = roster_job() | {"payload": {}}
    assert client.post("/jobs", json=no_employee, headers=ACME).status_code == 422


def test_duplicate_job_is_409(client):
    client.post("/jobs", json=roster_job(), headers=ACME)
    assert client.post("/jobs", json=roster_job(), headers=ACME).status_code == 409


def test_trigger_pause_resume_dead_letters_metrics(client):
    client.post("/jobs", json=roster_job(), headers=ACME)
    assert client.post("/jobs/roster/trigger", headers=ACME).status_code == 202
    assert client.post("/tenants/acme/pause", headers=ACME).json()["paused"] is True
    assert client.post("/tenants/acme/resume", headers=ACME).json()["paused"] is False
    assert client.get("/dead-letters", headers=ACME).json() == []
    m = client.get("/metrics", headers=ACME).json()
    assert {"running", "retrying", "dead_lettered", "fire_lag_histogram"} <= m.keys()
```

`requirements.txt`

```
fastapi==0.162.0
uvicorn==0.54.0
croniter==6.2.4
httpx==0.28.1
pytest==9.1.1
tzdata; sys_platform == "win32"
```